Utiliser les classes fournies : lire, écrire et modifier des tickets

420-249-AH - Programmation mobile iOS

1. Les éléments fournis

Le projet de départ contient les éléments suivants, à utiliser tels quels :

Élément	Rôle	Utilisation
`Ticket`	Représente un ticket de support	`Ticket(titre:...)`
`CategorieTicket`	Catégories possibles	`.materiel, .reseau, ...`
`PrioriteTicket`	Priorités possibles	`.basse, .haute, ...`
`StatutTicket`	Statuts possibles	`.nouveau, .enCours, ...`
`DonneesDemo`	12 tickets de démonstration	`DonneesDemo.tickets()`
`JSONStore`	Charge et enregistre les tickets	`charger, sauvegarder`

Remarque : le fonctionnement interne de JSONStore sera étudié dans un prochain cours. Pour l'instant, on l'utilise comme un outil : on lui demande de charger ou de sauvegarder des tickets.

2. Créer un ticket

Un ticket se crée avec son titre, sa description, sa catégorie et sa priorité :

```
let ticket = Ticket(
    titre: "Imprimante hors service",
    description: "L'imprimante du 2e étage ne répond plus.",
    categorie: .imprimante,
    priorite: .haute
)
```

- Le statut vaut Nouveau par défaut
- L'identifiant (id) et la date de création (dateCreation) sont créés automatiquement
- .imprimante et .haute sont des valeurs des énumérations fournies

3. Lire les informations d'un ticket

Chaque information est une propriété du ticket :

```
print(ticket.titre)
print(ticket.categorie.libelle)
print(ticket.priorite.libelle)
print(ticket.statut.libelle)
```

Résultat :

```
Console
Imprimante hors service
Imprimante
Haute
Nouveau
```

- libelle donne le texte à afficher pour une valeur d'énumération
- description, id et dateCreation sont aussi disponibles

4. Les énumérations fournies

Chaque énumération donne la liste de ses valeurs (allCases) et un texte à afficher :

```
for categorie in CategorieTicket.allCases {
    print("\(categorie.libelle) : \(categorie.icone)")
}
```

Résultat :

```
Console
Matériel : desktopcomputer
Logiciel : apps.iphone
Réseau : wifi
Compte utilisateur : person.crop.circle
Imprimante : printer
Autre : ellipsis.circle
```

Énumération	Propriétés fournies
`CategorieTicket`	libelle, icone
`PrioriteTicket`	libelle (les priorités se comparent avec < et >)
`StatutTicket`	libelle, libellePluriel

```swift
for statut in StatutTicket.allCases {
    print(statut.libellePluriel)
}

print(PrioriteTicket.haute > PrioriteTicket.basse)
```

Résultat :

```
Console
Nouveaux
En cours
Résolus
Fermés
true
```

5. Les données de démonstration

DonneesDemo.tickets() retourne un tableau de tickets prêt à l'emploi :

```swift
let demo = DonneesDemo.tickets()
print("\(demo.count) tickets de démonstration")
print(demo[0].titre)
```

Résultat :

```
Console
12 tickets de démonstration
Problème Wi-Fi au 3e étage
```

Répartition des 12 tickets : 3 nouveaux, 4 en cours, 3 résolus et 2 fermés.

6. Charger les tickets

JSONStore lit les tickets enregistrés. On lui indique le nom du fichier et le type attendu, [Ticket].self :

```swift
let store = JSONStore(nomFichier: "tickets.json")
var tickets = store.charger([Ticket].self) ?? DonneesDemo.tickets()
print("\(tickets.count) tickets chargés")
```

Résultat :

```
Console
12 tickets chargés
```

charger retourne un optionnel :

Résultat	Signification	Action
`Un tableau`	Des tickets ont été enregistrés	Les utiliser
`nil`	Aucun fichier encore (premier lancement)	Utiliser les données de démonstration

L'opérateur ?? fournit la valeur de remplacement : ici, les données de démonstration.

7. Sauvegarder les tickets

sauvegarder enregistre le tableau de tickets dans le fichier :

```
store.sauvegarder(tickets)

let relus = store.charger([Ticket].self)
print("Tickets relus : \(relus?.count ?? 0)")
```

Résultat :

```
Console
Tickets relus : 12
```

- On sauvegarde le tableau complet, pas un seul ticket
- sauvegarder retourne true si l'enregistrement a réussi

8. Ajouter un ticket

On ajoute le ticket au tableau avec append, puis on sauvegarde :

```
let nouveau = Ticket(
    titre: "Souris défectueuse",
    description: "Le clic droit ne répond plus.",
    categorie: .materiel,
    priorite: .basse
)
tickets.append(nouveau)
store.sauvegarder(tickets)
print("\(tickets.count) tickets")
```

Résultat :

```
Console
13 tickets
```

9. Modifier un ticket

Pour modifier un ticket, on procède en trois étapes :

- Retrouver le ticket dans le tableau avec son identifiant
- Changer ses propriétés
- Sauvegarder le tableau

```swift
let identifiant = nouveau.id

if let index = tickets.firstIndex(where: { $0.id == identifiant }) {
    tickets[index].statut = .enCours
    tickets[index].priorite = .haute
}
store.sauvegarder(tickets)

let modifie = tickets.first { $0.id == identifiant }
print(modifie?.statut.libelle ?? "introuvable")
```

Résultat :

```
Console
En cours
```

- firstIndex(where:) retourne la position du ticket, ou nil s'il n'existe pas
- On modifie directement l'élément du tableau : tickets[index]

10. Attention : modifier une copie

Ticket est une structure : une variable qui reçoit un ticket en reçoit une copie. Modifier la copie ne modifie pas le tableau :

```swift
var copie = tickets[0]
copie.statut = .ferme
print(tickets[0].statut.libelle)

tickets[0] = copie
print(tickets[0].statut.libelle)
```

Résultat :

```
Console
En cours
Fermé
```

Important : après avoir modifié une copie, il faut la remettre dans le tableau (tickets[0] = copie), puis sauvegarder.

11. Supprimer un ticket

removeAll retire du tableau tous les tickets qui respectent la condition, ici le même identifiant :

```swift
tickets.removeAll { $0.id == identifiant }
store.sauvegarder(tickets)
print("\(tickets.count) tickets")
```

Résultat :

```
Console
12 tickets
```

12. Vérifier après une relance

Recharger les tickets depuis le fichier simule la fermeture puis la réouverture de l'application :

```swift
let apresRelance = store.charger([Ticket].self) ?? []
print("\(apresRelance.count) tickets après la relance")
print(apresRelance[0].statut.libelle)
```

Résultat :

```
Console
12 tickets après la relance
Fermé
```

Les modifications sont retrouvées : le premier ticket est toujours Fermé.

13. Où appeler ces opérations ?

Ces opérations ne s'écrivent pas dans les vues. On les place dans une classe du projet (le ViewModel) que les vues appellent :

```
Vue (SwiftUI)
      ↓
Votre classe (ViewModel)
      ↓
JSONStore
      ↓
Fichier tickets.json
```

- La vue appelle une méthode pour ajouter, modifier ou supprimer
- La classe garde le tableau de tickets, le charge au démarrage et le sauvegarde après chaque changement

14. Résumé

Opération	À retenir
`Créer`	Ticket(titre:description:categorie:priorite:)
`Lire`	Les propriétés du ticket ; libelle pour les énumérations
`Charger`	store.charger([Ticket].self) retourne un optionnel ; ?? pour les données de démonstration
`Sauvegarder`	store.sauvegarder(tickets) enregistre le tableau complet
`Ajouter`	append, puis sauvegarder
`Modifier`	firstIndex(where:), modifier tickets[index], puis sauvegarder
`Supprimer`	removeAll { }, puis sauvegarder